

ALGORITMUL DE BACKPROPAGATION PENTRU REȚELE NEURONALE FEED FORWARD

1. SCOPUL LUCRĂRII

În această lucrare se studiază algoritmul de back-propagation, pentru rețele neuronale și aplicații ale acestui algoritm.

2. BREVIAR TEORETIC

Faza cea mai importantă în proiectarea și realizarea unei rețele neuronale artificiale este învățarea (antrenarea). Cel mai cunoscut și cel mai aplicat algoritm de învățare în rețelele neuronale MLP este algoritmul de backpropagation. Descoperirea lui în anii '80 a determinat o retrezire masivă a interesului pentru cercetarea în rețele neuronale.

În faza de învățare, rețeaua neuronală este antrenată să învețe un număr de șabloane (perechi intrări-ieșiri dorite) cu o anumită eroare specificată. Numărul de șabloane necesare depinde de tipul și complexitatea problemei. Intrările din șablonul curent impun anumite valori ieșirilor rețelei neuronale (valori calculate în pasul *forward* al antrenării). Ieșirile calculate se compară cu vectorul de ieșire așteptat (cel dat de ieșirile șablonului curent). Diferența, reprezentând de fapt eroarea curentă, se propagă înapoi în rețea (pasul *backward* al antrenării) ajustându-se toate ponderile rețelei conform unor relații matematice ce asigură minimizarea erorii globale de învățare a întregului set de șabloane. Aceste relații de ajustare, sunt prezentate în detaliu, ca și comentarii, în listingul programelor prezentate.

Când rețeaua a învățat setul de șabloane cu precizia impusă, spunem ca rețeaua neuronală a convers. Viteza de convergență este o caracteristică importantă a rețelei neuronale utilizate într-o aplicație specifică. Ea depinde de factorii următori:

- arhitectura rețelei (numărul de straturi ascunse și numărul de neuroni din fiecare strat)
- numărul de șabloane de antrenare de învățat;
- eroarea globală de antrenare impusă;

L3 – Algoritmul de backpropagation pentru rețele neuronale feed forward

- parametrii aleși pentru algoritmul de backpropagation;
- ordinea de prezentare a șabloanelor.

Prezentarea întregului set de șabloane rețelei neuronale, o singură dată, cu ajustarea corespunzătoare a ponderilor determină o *epocă de antrenare*. Astfel, viteza de convergență a unei rețele neuronale se măsoară în numărul de epoci necesar pentru ca întregul set de exemple de antrenare să fie învățat cu eroarea dorită.

Forma cea mai generală a algoritmului de backpropagation este următoarea:

- se inițializează toate ponderile rețelei cu valori aleatoare mici

SE REPETĂ

Pentru fiecare șablon de antrenare se repetă

- 1) se prezintă vectorul de intrare, din perechea de antrenare curentă, la intrările rețelei neuronale și se propagă fluxul de activare, nivel cu nivel, până la ieșire (pasul *forward*).
- 2) se calculează eroarea pe baza ieșirilor obținute și a celor dorite (impuse de șablon) și se propagă această eroare înapoi, nivel cu nivel, până la intrare, ajustându-se ponderile (pasul *backward*).

PÂNĂ CÂND eroarea calculată scade sub limita impusă.

3. DESFĂȘURAREA LUCRĂRII

Se vor edita și apoi executa programele descrise în continuare.

3.1. Programul 1

Scrieți un program ce creează și antrenează o rețea ce trebuie să învețe funcția XOR, cu 2 intrări. Arhitectura rețelei: 2 neuroni în stratul de intrare, 2 neuroni în stratul ascuns, 1 neuron în stratul de ieșire.

Listingul programului

```
import java.util.*;  
class XOR_1{
```

/ Notatiile neuronilor din rețeaua 2-2-1, sunt:*

L3 – Algoritmul de backpropagation pentru rețele neuronale feed forward

#0 #2

#4

#1 #3

*/

```
public static void main(String args[])
{
    double eta=1.0;
    double alfa=1.0;
    double eroareCrtToateSabloanele;
    double eroareMaxToateSabloanele;
    double w20,w30,w21,w31,w42,w43;
    //w20 este ponderea conexiunii intre neuronii 0 si 2
    double out[]=new double[4];
    double yCalculat, yDorit;
    double sablonIn_0[]={0.1,0.1,0.9,0.9};//sabloanele de antrenare
                                         //pt. neuron 0.
    double sablonIn_1[]={0.1,0.9,0.1,0.9};
    double sablonOut[] ={0.1,0.9,0.9,0.1};//pentru functia XOR
    double delta2,delta3,delta4;
    //delta2=semnalul delta calculat pentru neuronul 2
    double eroareCrtUnSablon[]=new double[4];
    int nrCrtEpoci;
    boolean converged=false;
    int i,c;
    final double EROARE_LIMITA=0.01;

    Random r=new Random();
    w20=r.nextDouble()-0.5;
    w30=r.nextDouble()-0.5;
    w21=r.nextDouble()-0.5;
    w31=r.nextDouble()-0.5;
    w42=r.nextDouble()-0.5;
    w43=r.nextDouble()-0.5;

    nrCrtEpoci=0;
```

L3 – Algoritmul de backpropagation pentru rețele neuronale feed forward

```
//Algoritmul de backpropagation:
while(!converged){
  for(i=0;i<4;i++){//i-este nr. sablonului
    //propagare inainte:
    out[0]=sablonIn_0[i];
    out[1]=sablonIn_1[i];
    out[2]=1/(1+Math.exp(-w20*out[0]-w21*out[1]));
    out[3]=1/(1+Math.exp(-w30*out[0]-w31*out[1]));
    yDorit=sablonOut[i];
    yCalculat=1/(1+Math.exp(-w42*out[2]-w43*out[3]));
    eroareCrtUnSablon[i]=0.5*(yCalculat-yDorit)*(yCalculat-
      yDorit);
    //modifica ponderi:
    //Formula de ajustare ponderi: w_ji=alfa*w_ji - eta*out[i]*delta_j
    //Daca j este nod din ultimul strat:
    //delta_j=(yCalculat_j-yDorit_j)*derivata functiei de transfer
    delta4=(yCalculat-yDorit)*(1-yCalculat)*yCalculat;
    w42=alfa*w42-eta*out[2]*delta4;
    w43=alfa*w43-eta*out[3]*delta4;
  /*
    Daca j este nod din strat ascuns:
    delta_j = (suma ponderata delta noduri fii ai lui j)*derivata functiei
    de transfer in raport cu intrarea totala, calculata in nodul j.
    Pentru functia de transfer sigmoida :
    f=1/(1+exp(-x)), avem: derivata f'(x)=f(1-f).
    Deci in nodul j, derivata este: out[j]*(1-out[j]) .
  */
    delta2=w42*delta4*(1-out[2])*out[2];
    delta3=w43*delta4*(1-out[3])*out[3];
    w20=alfa*w20-eta*out[0]*delta2;
    w30=alfa*w30-eta*out[0]*delta3;
    w21=alfa*w21-eta*out[1]*delta2;
    w31=alfa*w31-eta*out[1]*delta3;
  }//for
  nrCrtEpoci++;
  eroareCrtToateSabloanele=0;
  for(i=0;i<4;i++){
    eroareCrtToateSabloanele+=eroareCrtUnSablon[i];
  }
}
```

L3 – Algoritmul de backpropagation pentru rețele neuronale feed forward

```
System.out.println("Epoca: "+nrCrtEpoci+". Eroarea totala: "
+eroareCrtToateSabloanele);
if(eroareCrtToateSabloanele<=EROARE_LIMITA)
    converged=true;
}while
//afisare ponderi:
System.out.println("w20="+w20+" w21="+w21);
System.out.println("w30="+w30+" w31="+w31);
System.out.println("w42="+w42+" w43="+w43);
} //main
} //class
```

Tema

- Rulați de mai multe ori programul. De ce se obțin de fiecare dată alte valori pentru ponderi ?
- Rulați programul și cu alte valori de EROARE_LIMITA
- Modificați programul astfel încât:
 - rețeaua să învețe funcția OR
 - EROARE_LIMITA să se citească de la tastatură
 - Antrenarea să se poată opri forțat, prin apăsarea tastei ESC
 - Ponderile găsite, dacă antrenarea a convers, să fie salvate într-un fișier text.
 - să permită antrenarea în trepte a rețelei. (Se specifică la început o eroare mai mare, iar după ce a convers se specifică o eroare mai mica, folosind ponderile găsite la treapta precedentă, ca ponderi inițiale ale rețelei).

3.2. Programul 2

Scrieți un program ce creează și antrenează o rețea ce trebuie să învețe funcția XOR, cu 2 intrări. Arhitectura rețelei: 3 neuroni în stratul de intrare, 2 neuroni în stratul ascuns, 1 neuron în stratul de ieșire.

Listingul programului

/*

Notatiile neuronilor din retea:

#0

#2

L3 – Algoritmul de backpropagation pentru rețele neuronale feed forward

#1 #3 #5
 #4

*Neuronii 0 si 1 sunt neuroni de intrare.
Neuronii 2,3,4 sunt neuronii stratului ascuns.
Neuronul 5 este neuronul din stratul de iesire.
w20 este ponderea intre neuronii 0 si 2
w30 este ponderea intre neuronii 0 si 3
w40 este ponderea intre neuronii 0 si 4
w21 este ponderea intre neuronii 1 si 2
w31 este ponderea intre neuronii 1 si 3
w41 este ponderea intre neuronii 1 si 4
w52 este ponderea intre neuronii 2 si 5
w53 este ponderea intre neuronii 3 si 5
w54 este ponderea intre neuronii 4 si 5*

```
*/
import java.util.*;
class XOR_2{
public static void main(String args[])
{
    final double EROARE_LIMITA=0.001;
    double eta=1.0;
    double alfa=1.0;
    double eroareCrtToateSabloanele;
    double eroareMaxToateSabloanele;
    double w20,w30,w40,w21,w31,w41,w52,w53,w54;
    double out[]=new double[5];//iesirile neuronilor 0..4
    double yCalculat, yDorit;//iesirea neuronului 5
    //sabloanele de antrenare:
    double sablonOut[]={0.1,0.9,0.9,0.1};
    double sablonIn_0[]={0.1,0.1,0.9,0.9};//pentru neuronul 0.
    double sablonIn_1[]={0.1,0.9,0.1,0.9};

    //Valorile delta calculate de algoritmul de backpropagation,
    // in nodurile 2,3,4, 5:
    double delta2,delta3,delta4,delta5;
    double eroareCrtUnSablon[]=new double[4];
```

L3 – Algoritmul de backpropagation pentru rețele neuronale feed forward

```
int nrCrtEpoci;
boolean converged=false;
int i;//numarul curent al sablonului (exemplului de antrenare)
int c;
Random r=new Random();
w20=r.nextDouble()-0.5;
w30=r.nextDouble()-0.5;
w40=r.nextDouble()-0.5;
w21=r.nextDouble()-0.5;
w31=r.nextDouble()-0.5;
w41=r.nextDouble()-0.5;
w52=r.nextDouble()-0.5;
w53=r.nextDouble()-0.5;
w54=r.nextDouble()-0.5;
nrCrtEpoci=0;
//Algoritmul de backpropagation:
while(!converged){
for(i=0;i<4;i++){
//propagare inainte:
out[0]=sablonIn_0[i];
out[1]=sablonIn_1[i];
out[2]=1/(1+Math.exp(-w20*out[0]-w21*out[1]));
out[3]=1/(1+Math.exp(-w30*out[0]-w31*out[1]));
out[4]=1/(1+Math.exp(-w40*out[0]-w41*out[1]));
yDorit=sablonOut[i];
yCalculat=1/(1+Math.exp(-w52*out[2]-w53*out[3]-
w54*out[4]));
eroareCrtUnSablon[i]=0.5*(yCalculat-yDorit)*(yCalculat-
yDorit);
//modifica ponderi:
//(eta=alfa=1.0)
//Eroarea calculata de algoritmul de backpropagation in
// nodul de iesire, delta5:
delta5=(yCalculat-yDorit)*(1-yCalculat)*yCalculat;
//Ajustarea ponderilor conexiunilor la nodul 5:
w52=alfa*w52-eta*out[2]*delta5;
w53=alfa*w53-eta*out[3]*delta5;
w54=alfa*w54-eta*out[4]*delta5;
```

L3 – Algoritmul de backpropagation pentru rețele neuronale feed forward

```
//Pentru neuronul (nodul) 2:
delta2=w52*delta5*(1-out[2])*out[2];
//In nodul 3:
delta3=w53*delta5*(1-out[3])*out[3];
//In neuronul 4:
delta4=w54*delta5*(1-out[4])*out[4];
//Ajustarea ponderilor conexiunilor conectate la nodurile 2, 3 si
4:
w20=alfa*w20-eta*out[0]*delta2;
w30=alfa*w30-eta*out[0]*delta3;
w40=alfa*w40-eta*out[0]*delta4;
w21=alfa*w21-eta*out[1]*delta2;
w31=alfa*w31-eta*out[1]*delta3;
w41=alfa*w41-eta*out[1]*delta4;
} //for
nrCrtEpoci++;
eroareCrtToateSabloanele=0;
for(i=0;i<4;i++)
    eroareCrtToateSabloanele+=eroareCrtUnSablon[i];
System.out.println("Epoca: "+nrCrtEpoci+
    "
    Eroarea
totala:"+eroareCrtToateSabloanele);
if(eroareCrtToateSabloanele<=EROARE_LIMITA)
    converged=true;
} //while
//afisare ponderi:
System.out.println("w20="+w20+" w21="+w21);
System.out.println(" w30="+w30+" w31="+w31);
System.out.println("w40="+w40+" w41="+w41);
System.out.println("w52="+w52+"
    w53="+w53+"
w54="+w54);
} //main
} //class
```

Tema

- Comparați viteza de convergență la învățarea funcției XOR, între cele 2 arhitecturi de rețele feed forward (cea din programul 1 și cea din programul curent).

L3 – Algoritmul de backpropagation pentru rețele neuronale feed forward
